本科生毕业论文（设计）

题目：基于强化学习的游戏 AI Agent 研究：以俄罗斯方块为例

姓　　名　徐俊萍

学　　号　20350069

院　　系　哲学系

专　　业　逻辑学

指导教师　鲜于波教授

2025 年 5 月 16 日

基于强化学习的游戏 AI Agent 研究：以俄罗斯方块为例

Reinforcement Learning for Game AI Agents: A Tetris Case Study

姓　　名　徐俊萍

学　　号　20350069

院　　系　哲学系

专　　业　逻辑学

指导教师　鲜于波教授

2025 年 5 月 16 日

学术诚信声明

本人郑重声明：所呈交的毕业论文（设计），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文（设计）不包含任何其他个人或集体已经发表或撰写过的作品成果。对本论文（设计）的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本论文（设计）的知识产权归属于培养单位。本人完全意识到本声明的法律结果由本人承担。

作者签名：

日　　期：　　年　　月　　日

摘要

本研究致力于应对经典游戏俄罗斯方块（Tetris）所呈现的复杂序贯决策挑战，旨在通过深度强化学习（Deep Reinforcement Learning,DRL）技术开发高性能的人工智能体（AI Agent）。针对俄罗斯方块固有的随机性、高维状态空间以及即时奖励与长远目标间的权衡难题，本研究选用 Double Deep Q-Network (DDQN) 算法以缓解标准 DQN 中的 Q 值过高估计偏差。研究的核心创新在于采用基于领域知识的特征工程方法构建状态表征，将棋盘状态抽象为包含潜在消除行数、空洞数量、表面崎岖度及累积高度的四维特征向量，并输入至多层感知机（Multi-Layer Perceptron, MLP）进行价值函数逼近，替代了计算成本较高的卷积神经网络（Convolutional Neural Networks, CNN）。动作空间被定义为最终落子状态（水平位置与旋转姿态）。奖励函数经精心设计，通过设置生存奖励、对消除行数施加二次方放大效应以及对游戏结束状态施加惩罚，以有效引导 Agent 学习兼顾生存与高效得分的策略。在定制的模拟环境中，通过 3000 回合的训练与 ε-greedy 探索策略，Agent 展现出显著的学习能力和性能提升，实现了单回合最高消除 468 行的记录。实证结果验证了所提出方法的有效性，证明结合特征工程的 DDQN 能够高效地解决俄罗斯方块问题，并为资源受限环境下训练高性能游戏 AI 提供了实践参考。论文最后探讨了该方法的优势、潜在局限性，并对未来研究方向，如算法融合、表征学习及跨范式比较等进行了展望。

关键词：强化学习，AI Agent，俄罗斯方块，Double DQN，游戏 AI

ABSTRACT

This research addresses the complex sequential decision-making challenges inherent in the classic game Tetris, aiming to develop a high-performance Artificial Intelligence (AI) agent utilizing Deep Reinforcement Learning (DRL). To tackle the game's stochastic nature, high-dimensional state space, and the critical trade-off between immediate rewards and long-term objectives, this study employs the Double Deep Q-Network (DDQN) algorithm, specifically chosen to mitigate the Q-value overestimation bias common in standard DQN. A core contribution lies in the adoption of a domain knowledge-driven feature engineering approach for state representation. Instead of computationally expensive Convolutional Neural Networks (CNNs), the board state is abstracted into a four-dimensional feature vector—comprising potential lines cleared, hole count, surface bumpiness, and aggregate height—which serves as input to a Multi-Layer Perceptron (MLP) for value function approximation. The action space is defined by the final tetromino placement state (horizontal position and rotation). A carefully designed reward function, incorporating survival incentives, a quadratic amplification for cleared lines, and penalties for game termination, effectively guides the agent towards strategies balancing longevity and scoring efficiency. Empirical evaluation through 3000 training episodes within a custom simulation environment, employing an ε-greedy exploration strategy, demonstrated significant learning efficacy and performance gains, culminating in a peak score of 468 cleared lines in a single episode. These results validate the effectiveness of the proposed feature-based DDQN methodology for Tetris, offering practical insights into developing proficient game AI under potentially resource-constrained conditions. The paper concludes by discussing the advantages and limitations of this approach and outlining future research avenues, including algorithmic enhancements, representation learning, and cross-paradigm comparisons.

Keywords: Reinforcement Learning, AI Agent, Tetris, Double DQN, Game AI

目录

1 引言 1
1.1 背景与动机 1
1.2 文献综述与方法定位 2
2 模型设定与实验设计 5
2.1 搭建俄罗斯方块模拟环境 5
2.2 状态表示 6
2.3 动作空间 6
2.4 Double DQN 结构 7
2.5 Double DQN 算法实现 8
2.6 奖励函数 10
2.7 实现工具 11
3 实验与结果 12
3.1 实验设置 12
3.2 学习数据分析 12
4 讨论 17
4.1 实验结果分析与方法有效性评估 17
4.2 基于实验结果的方法比较与讨论 18
5 未来工作 20
参考文献 21
致谢 22

1 引言

1.1 背景与动机

人工智能（Artificial Intelligence, AI）的快速发展正以前所未有的方式改变着世界，而游戏作为一种模拟现实世界、规则明确且具有挑战性的环境，一直以来都是检验和推动 AI 技术进步的重要试验场。从早期国际象棋中的 IBM “深蓝”展现搜索算法的威力，Google DeepMind 的 AlphaGo 融合深度学习与强化学习，在围棋领域击败人类世界冠军，[1] 再到近期的 Gemini Pro 直播通关宝可梦，以及众多个人研究团队通过上下文强化学习[2]与离线强化学习[3]构建大语言模型智能体，在宝可梦实际对战中与人类玩家一较高下。AI 在游戏领域取得的诸多成就，也为现实世界中处理复杂决策，优化解决问题提供了崭新视角。

本研究聚焦于俄罗斯方块（Tetris）这一风靡全球的经典益智游戏，其基本规则是：玩家操纵从屏幕顶部随机下落的，具有不同形状的积木块（Tetrominoes），通过旋转和水平移动操作，将其放置到底部区域，目的是填充并消除尽可能多的横行，阻止或减缓积木堆叠至顶端，导致游戏结束。然而，在这看似简单的规则下，俄罗斯方块自身却具有显著的策略复杂性，考验着玩家的快速反应与空间规划能力，为其设计高效的 AI Agent 更是一项颇具挑战性的尝试。首先，游戏的输入具有随机性。每次下落的积木块形状是不确定的，这要求 Agent 具备良好的泛化能力，及对未知情况的处理能力。其次，尽管棋盘规模固定（通常为 `10x20`），但理论上来说，每个格子都具有空闲或被占据两种状态，因而整个棋盘可能的状态组合数量庞大（2^{200}），使得基于穷举搜索或状态遍历的传统方法难以为继。最后，优秀的俄罗斯方块策略需要权衡即时收益（如消除当前行）与长期目标（维持棋盘低位，为后续操作留出空间以获得持续的高分），简单的贪心策略往往会导致棋盘迅速堆叠，尽早结束游戏，因而 Agent 必须在即时收益与长期存活之间寻找平衡点。

从抽象意义上来看，俄罗斯方块可以被视为一类具有随机性和巨大状态空间的序贯决策问题 (Sequential Decision-Making Problem)，与驾驶汽车、经营公司类似，拥有巨大的状态与动作空间，与环境之间存在动态交互，过程具有随机性或不确定性，以及奖励延迟性与长期依赖等特点。而强化学习（Reinforcement Learning, RL）为解决此类序贯决策问题提供了合适的理论框架。在 RL 范式下，Agent 通过不断与环境交互，根据获得的奖励（如消除行数）或惩罚（如游戏结束）信号调整自身的行为策略，使得长期累积奖励最大化。随着深度学习（Deep Learning, DL）技术的快速成熟，尤其是深度神

经网络（Deep Neural Network, DNN）强大的特征提取和函数逼近能力，使得深度强化学习（Deep Reinforcement Learning, DRL）应运而生。DRL 成功地将 DNN 与 RL 相结合，使得 Agent 能够直接从高维原始输入（如图像）中学习复杂的控制策略，并在诸多领域取得了突破性进展[4]，为解决俄罗斯方块这类具有复杂状态空间的问题提供了切实可行的路径。

1.2 文献综述与方法定位

针对俄罗斯方块设计 AI Agent 的研究由来已久，涵盖了多种方法。本节将回顾这些相关工作，并在此基础上阐述本研究的定位。

1.2.1 早期俄罗斯方块 AI 研究

早期，针对俄罗斯方块 AI 的研究主要依赖于专家知识和手工设计的启发式评估函数（Heuristic Evaluation Functions）以及策略迭代（Policy Iteration）方法。Dellacherie 通过调整特征权重构建了著名的启发式函数，在当时的 Tetris AI 领域具有极高水平[1]。Bertsekas 和 Tsitsiklis 利用近似策略迭代方法也获得了显著成果[1]。另一类研究者则使用进化算法（Evolutionary Algorithms, EA），通过优化预定义状态特征的权重，来量化棋盘状态的优劣。例如，Böhm 等人通过优化诸如空洞数量、方块堆叠高度、平整度等特征的权重来寻找最优组合[1]。这些早期方法往往依赖于人工设计的特征和规则，在处理复杂和动态环境时或存在局限性。

1.2.2 深度强化学习的应用与方法选择

深度强化学习（DRL）的兴起，为解决复杂决策问题带来了新的突破。Mnih 等人提出的深度 Q 网络（Deep Q-Networks, DQN）算法[4] 在 DRL 领域中具有开创性意义，成功地将卷积神经网络（Convolutional Neural Network, CNN）与 Q-Learning 相结合，使得 Agent 能够直接从高维原始输入（如游戏像素画面）学习控制策略，并通过经验回放（Experience Replay）和目标网络（Target Network）等技术有效稳定了训练过程，在 Atari 游戏（涵盖多种游戏类型，如动作、射击、运动、赛车等，常被用作 AI 训练基准）上取得了超越人类专家的表现成果。

标准的 Q-Learning 及 DQN 算法存在 Q 值高估（Overestimation Bias）的问题[4,6]。其根源在于：使用同一个 Q 网络来选择和评估下一个状态的最佳动作，这种“自己选自己评”的操作，会强选择那些被偶然高估的价值，使得最终的价值估计偏高累计，进而使得训练不稳定或者收敛到次优策略。为解决这一问题，Van Hasselt 提出了 Double

Q-Learning 算法[4]，通过解耦动作选择与价值评估，有效减轻了过高估计的问题。在此基础上，Van Hasselt 等人将 Double Q-Learning 的思想应用于 DQN 框架，提出了 Double DQN（DDQN）算法[5]，进一步提高了模型的性能与学习过程的稳定性。

除了 DDQN 方法，DRL 领域还有许多重要的技术改进，如优先经验回放（Prioritized Experience Replay, PER)[7]、决斗网络架构（Dueling Network Architectures)[8]、多步学习（Multi-step Learning)、分布式强化学习（Distributional RL)、噪声网络（Noisy Nets)、分布式架构 (Distributed Architectures) 等。Hessel 等人提出的 Rainbow Agent[6] 就整合了其中的多项技术，并展示了它们强大的协同效应。

1.2.3 其它相关方法

进化策略（Evolutionary Strategies, ES）是另一类适用于优化控制策略的黑盒优化算法[9]，不依赖于梯度信息，也不需要价值函数，通过参数空间的随机扰动（变异），进行评估与选择，优化迭代目标函数[9]。ES 也曾被应用于 Tetris AI 的算法构建，如 Boumaza 使用 ES 并结合 Racing 机制（动态调整评估次数以提高效率）来设计 Tetris 玩家[10]，通过置信区间判断个体优劣，避免了对每个个体进行固定次数的评估，从而加速了进化过程。尽管其在某些方面具有吸引力，但通常认为其在样本效率上低于基于梯度的强化学习方法[10]。

启发式搜索方法 (Heuristic Search) 结合评估函数，在每一步动作之前，显式地向前“看几步”，以搜索最佳动作序列。这种方法的优点是决策过程可解释性更强，并且可以通过调整搜索深度或宽度来平衡计算成本与决策质量。缺点在于它严重依赖于启发式函数的质量，并且每一步决策都需要进行搜索计算。Da Col 和 Teppan 就在 Tetris 中应用了启发式搜索 (beam search)[1]，其研究表明，即使存在“好”的启发式函数，增加搜索深度也存在一个“甜点”（sweet spot），过度搜索带来的收益反而会依次递减。

1.2.4 研究定位与预期贡献

综合以上文献回顾，针对俄罗斯方块复杂的状态空间和序贯决策特性，深度强化学习提供了一个有力的算法框架。考虑到标准 DQN 存在的 Q 值过高估计问题以及 Double DQN 在此方面的改进，本研究选择 Double Deep Q-Network (DDQN) 作为核心算法。

与许多直接处理高维原始像素输入的 DRL 工作不同[4]，本研究采用基于特征工程的状态表示。实验没有选择复杂的卷积神经网络（Convolutional Neural Network，CNN），而是将一组描述棋盘关键状态信息的低维特征（预测消除行数、空洞数、棋盘崎岖度与总高度）输入到相对简单的多层感知机（Multi-Layer Perceptron, MLP）结构的 Q 网络

中，以期利用人类的领域知识，以更低的计算资源与成本和更高的样本效率帮助 Agent 聚焦于与任务目标最相关的状态信息，从而有效学习策略。

本研究的主要研究内容将包括：

- 搭建功能完备的俄罗斯方块模拟环境，支持强化学习训练。
- 设计并实现基于四维特征向量的状态表示方法，用于捕捉俄罗斯方块棋盘的关键信息。
- 实现并训练了基于特征输入的 Double DQN AI Agent，并将其应用于俄罗斯方块策略的学习。
- 评估该 Agent 在 3000 个训练回合中的学习效果和性能，分析其消行数、步数、损失等关键指标的变化趋势。
- 结合实验结果，讨论特征表示、Double DQN 算法在 Tetris 任务中的有效性与局限性，并与相关研究进行比较。

2 模型设定与实验设计

本章详细介绍本研究中用于构建和训练俄罗斯方块 AI Agent 的方法和实现细节，包括游戏环境的搭建、Agent 的状态表示、动作空间、Double DQN 网络的结构以及训练算法流程。

2.1 搭建俄罗斯方块模拟环境

为支撑强化学习训练，我们基于 Python 的 pygame 库，搭建了一个功能完备的俄罗斯方块模拟环境，并封装在 Tetris 类中。该环境精确模拟了游戏的核心机制。游戏棋盘采用二维列表表示，由常量 WIDTH (宽度 =10) 和 HEIGHT (高度 =20) 定义，列表中，0 代表空格，非零整数（1 到 7 ）则对应被不同类型积木块占据的格子。七种标准俄罗斯方块的形状及其对应的渲染颜色分别在 Tetris.pieces 和 Tetris.piece_colors 列表中定义。

游戏的核心机制通过一系列方法实现。rotate(piece) 实现了积木块的旋转。check_collision(piece, pos) 用于检测给定积木块在指定位置 (pos，包含 x，y 坐标) 是否与棋盘边界或已固定积木块发生碰撞，这是判断移动或旋转是否有效以及积木块是否落地的关键函数。当积木块完成下落后，store(piece, pos) 函数会将其形状和类型（由 self.ind 标识）固定到棋盘状态的副本上，值函数返回修改后的棋盘副本，并不直接修改环境的内部棋盘状态 self.board。满行检测与消除则由 check_cleared_rows(board) 和remove_row(board, indices) 协同完成，前者识别出完全填满的行，后者则移除这些行并将上方的方块下移。新积木块的生成则由 new_piece() 处理，该方法采用了”7-bag” 随机化机制（通过 random.shuffle(self.bag) 实现），以保证各种积木块类型的出现相对均衡。新块生成后会立即检查其在初始位置是否发生碰撞，若发生碰撞则表明游戏结束，设置 self.gameover 标志。

对于强化学习流程，环境提供了 reset() 和 step(action, render) 接口，对游戏状态进行重置与步进。reset() 方法用于初始化游戏状态，包括清空棋盘、重置得分和行数计数器，并生成第一个积木块，返回棋盘的初始状态特征。step(action, render) 方法则执行 Agent 选择的动作 action（包含目标 x 坐标和旋转次数）。首先模拟积木块的旋转和平移，然后执行 “硬下落”（instant drop，因参数 DROP_SPEED 设置为 0）到最终位置。在这之后将积木块固定到棋盘上 (store)，(check_cleared_rows, remove_row) 方法用于检查并消除满行，从而根据消除的行数计算奖励值 reward，更新游戏统计数据（得分、已消除行数、已放置块数）。若游戏尚未结束，则生成下一个积木块 (new_piece)；若

新的木块生成即发生碰撞，或者先前的步骤导致堆叠溢出，则游戏结束，设置 gameover 标志。step 方法最终返回本轮游戏的奖励值 reward，以及确定游戏是否结束的布尔值 done。

2.2 状态表示

本研究采用了基于特征工程的状态表示方法，棋盘的状态特征由 get_state_properties(board) 函数计算得出，该函数接收一个棋盘状态（二维列表）作为输入，并返回一个包含四个浮点数的 PyTorch 张量。这四个特征具体为：

1. **当前状态下的潜在消除行数：**该特征并非直接计算当前棋盘能消除的行数，而是在 get_state_properties 内部调用 check_cleared_rows 时，模拟当前积木硬下落到所有可能的合法位置，同时计算该步所能消除的行数。这一特征旨在发现当前积木块的最佳即时收益潜力。
2. **空洞数：**通过 get_holes(board) 函数计算。空洞首先是一个空的格子，并且在其所在列的上方存在被占据的格子。空洞数是衡量棋盘状态优劣的常用指标，较少的空洞通常意味着更规整、更可持续的游戏状态。
3. **棋盘崎岖度：**通过 get_bumpiness_and_height(board) 函数计算。该函数首先确定棋盘每一列的方块堆叠高度（从最高被占据的格子到棋盘底部的距离），然后计算所有相邻列之间高度差的绝对值之和。崎岖度反映了棋盘表面的平整程度，数值越低越有利于放置新积木和消除行。
4. **总高度：**同样由 get_bumpiness_and_height(board) 函数计算，它是棋盘所有列的堆叠高度总和。总高度直接关联积木堆叠的整体水平，显而易见的是，过高的堆叠会增加游戏结束的风险。

选择这组特征是基于俄罗斯方块 AI 领域中常用的启发式规则，能够有效捕捉与游戏目标（最大化消行、维持低高度、保持平整、避免空洞等）直接相关的棋盘结构信息。同时，使用 MLP 代替 CNN，是考虑到与高维像素输入相比，使用低维特征向量能够显著降低对 Q 网络复杂度的要求，并且有望通过提供结构化的、与任务目标强相关的信息来加速学习过程。

2.3 动作空间

对于给定的当前积木块（self.piece）和棋盘状态（self.board），接下来 Agent 需要决定如何放置该积木块。理论上来说，可执行的动作包括一系列的左移、右移和旋转操作。然而，为了简化决策过程并与强化学习框架兼容，本研究将动作空间定义为积

木块最终的放置状态，具体由其目标水平位置（x 坐标）和旋转次数（num_rotations）来确定。

get_next_states() 函数负责枚举给定当前积木块和棋盘状态下，所有可能的有效动作。首先根据当前积木块的类型确定其所有有效的旋转次数（'O' 型块只有 1 种有效旋转状态，'I'、'S'、'Z' 型块有 2 种，其余有 4 种）。随后，对于每种有效的旋转状态，函数遍历所有可能的水平位置 x。在每个 (x, rotation) 组合下，模拟积木块的硬下落过程，直到检测到碰撞 (check_collision)，从而确定最终的垂直位置 (y_final)。若该最终位置有效（y_final >= 0），则模拟将该旋转后的积木块放置在 (x, y_final)，得到放置后的临时棋盘状态（通过 store 函数返回的副本）。接着，调用 get_state_properties 函数计算临时棋盘状态的 4 维特征向量，储存在字典中。最终，get_next_states() 返回该字典，其键是所有合法的动作 (x, num_rotations)，值为执行该动作后对应的下一状态特征张量。

在决策过程中，Agent 会利用 Q 网络评估 get_next_states() 返回的所有可能下一状态（由特征向量表示）的预期回报（Q 值），然后根据 ε-greedy 策略选择一个动作 (x, num_rotations) 来执行。

2.4 Double DQN 结构

本研究采用的 Q 网络是一个多层感知机（MLP），具体定义在 DoubleQNetwork 类中。其结构包括：

1. **输入层：** 接收 4 维的浮点数张量，对应 2.2 节中描述的四个状态特征。
2. **隐藏层：** 包含三个全连接（nn.Linear）隐藏层，每层有 64 个神经元。每个隐藏层各自后接一个 ReLU (Rectified Linear Unit, $f(x) = \max(0, x)$) 激活函数 (nn.ReLU)，用于引入非线性。
3. **输出层：** 本网络的输出层 (self.fc4) 为单一节点的全连接层，输出标量。基于本研究的状态表示和动作空间设计，该网络 $Q_{\text{net}}(s'; \theta)$ 的输出被用于估计 Agent 执行某个动作到达下一状态 s' 后所能获得的价值。更详细的解释将在 2.5 节的算法实现部分阐述。

此外，为了促进网络的稳定训练，本研究在 _create_weights 方法中对网络权重进行了初始化设置：线性层的权重采用 Xavier 均匀初始化 (nn.init.xavier_uniform_)，偏置项则初始化为 0 (nn.init.constant_)。

2.5 Double DQN 算法实现

完整的 Double DQN 训练流程由 Agent 类和全局 train 函数协同管理。Double DQN 算法旨在解决标准 DQN 中由于使用同一网络进行动作选择和价值评估而导致的 Q 值过高估计（Overestimation）问题。它通过引入一个独立的目标网络来评估下一状态的价值，从而提高学习的稳定性和性能。本研究的实现基于 Double DQN 的核心思想，并结合了特定的状态表示和动作空间设计。图 2.1 展示了训练流程的整体框架。

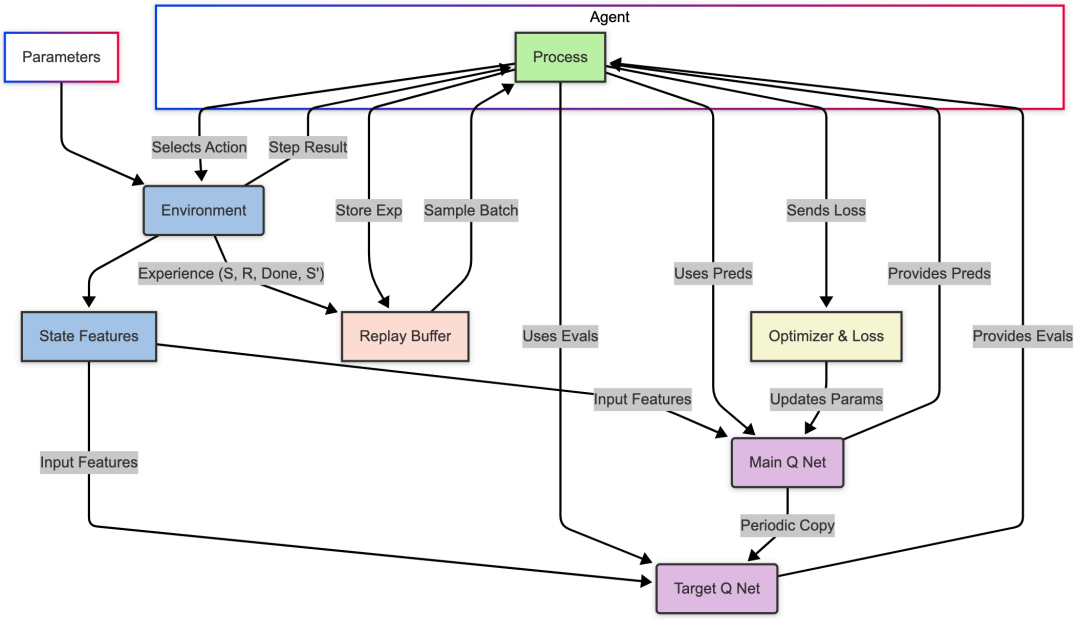


图 2.1　Double DQN 算法训练流程图

关键组件和步骤如下所述：

首先，在 Agent 类的初始化方法 __init__ 中，创建主网络（main_network）和目标网络（target_q_network），两者结构相同（均为 DoubleQNetwork 实例），并把它们移至指定设备（CPU 或 GPU）。主网络 θ 用于 Agent 的决策（通过评估潜在下一状态的价值）和进行参数更新，而目标网络 θ^- 则用于计算训练时的目标 Q 值。在这之后，目标网络立即设置为评估模式 (.eval())，其参数不会通过梯度下降直接更新。后续对目标网络参数进行初始化设置，使其与主网络保持一致 (update_target)。同时初始化经验回放缓冲区 replay_memory（使用 collections.deque 实现，其最大容量为 30000），用于存储 Agent 与环境交互产生的“经验”元组 $(s, a, r, s', \text{done})$。最后设置其他超参数，如训练的 minibatch 大小 BATCH_SIZE (512)、折扣因子 GAMMA (0.99) 以及目标网络更新

频率 update_target_rate (每 10000 总训练步数) 等。

为了使 Agent 在即时收益与长期存活之间寻找平衡，并兼顾探索未知状态，我们采用了 ε-greedy 策略。calc_epsilon(epoch) 函数根据当前训练的回合数（epoch）计算探索率 ε。在 3000 个回合的训练过程中，ε 从初始值 INITIAL_EPSILON (1.0) 开始，在 DECAY_EPOCHS 1500 个回合中，线性衰减至 FINAL_EPSILON (0.001)，最终维持 FINAL_EPSILON 这一探索率至训练结束。

在 train 函数内，每个游戏步骤都需要 Agent 执行以下动作选择过程。对于当前游戏状态 s_t，Agent 首先调用 env.get_next_states() 获取所有可能的最终放置动作集合 A_t 以及执行这些动作后对应的下一状态特征集合 $\{s'_{t,a} \mid a \in A_t\}$。然后，Agent 利用主网络 θ 评估每个可能的下一状态的价值，即计算 $\{Q_{\text{net}}(s'_{t,a};\theta) \mid a \in A_t\}$。在 ε-greedy 策略下，Agent 以 ε 的概率从 A_t 中随机选择一个动作执行；以 $1-\varepsilon$ 的概率选择能够最大化 Q_{net} 输出的下一状态所对应的动作 a^*:

$$a^* = \underset{a \in A_t}{\operatorname{argmax}}\, Q_{\text{net}}(s'_{t,a};\theta)$$

选定的动作 $a_t = (x, \text{num_rotations})$ 以及其对应的下一状态特征 s'_t 被确定下来。

训练过程的主体是 train 函数中的循环结构。外层循环遍历 EPOCHS 总共 3000 个回合。每个回合开始，环境通过 env.reset() 重置，并获取初始状态 s_0。内层循环则模拟一个完整的游戏回合，持续进行直到游戏结束 (done == True)。在内层循环的每一步中，执行上述动作选择过程，后在环境中执行选定的动作 a_t，环境返回奖励 r_t 和下一状态 s'_t（这里的 s'_t 实际上是环境经过内部处理（包括消行等）后，提供给 Agent 用于下一轮决策的状态特征），以及游戏是否结束的标志 done_t。Agent 将经验元组 $(s_t, a_t, r_t, s'_t, \text{done}_t)$ 存储到经验回放缓冲区 replay_memory 中。当前状态随后更新为 $s_{t+1} = s'_t$，并累积该步的奖励和步数。

当经验回放缓冲区中的样本数量超过 BATCH_SIZE（512）时，Agent 将会执行学习步骤。首先，从 replay_memory 中随机采样 minibatch (agent.get_minibatch)。接下来进行本过程的核心步骤目标值 y_batch 的计算。对于 minibatch 中的每一个下一状态 s'_j，使用主网络 θ 预测后续所有可能动作的 Q 值，并确定最大 Q 值 $Q(s'_j, a';\theta)$ 的动作索引 a'_{max}。更准确地说，代码实现是直接针对采样到的 next_state_batch 进行操作。然后，使用目标网络 θ^- 来评估由主网络选中的“最佳”动作（或者可以描述为其导致的下一状态）的价值：$Q_{target} = Q(s'_j, a'_{max};\theta^-)$。同样，在代码的具体实现中，这是直接对 next_state_batch 使用目标网络计算 Q 值，然后使用 .gather() 根据主网络确定的最佳动作索引，来选取对应的目标 Q 值。计算得到的 Q_{target} 用于构建 TD 目标：$y_j = r_j + \gamma \cdot Q_{target} \cdot (1 - done_j)$。

之后进行损失计算。损失函数采用均方误差 (MSELoss)，计算预测 Q 值与 TD 目标 y_j 之间的差距：

$$\text{Loss} = \frac{1}{\text{BatchSize}} \sum_j (y_j - Q(s_j, a_j; \theta))^2$$

计算出损失后，继续执行优化步骤：清零优化器梯度 (optimizer.zero_grad())，进行反向传播计算损失关于主网络参数 θ 的梯度 (loss.backward())，最后使用 Adam 优化器更新主网络参数 (optimizer.step())。

为了维持目标网络的稳定性，每隔 agent.update_target_rate (10000) 个总训练步数，通过调用 agent.update_target() 将主网络的参数整体复制到目标网络 ($\theta^- \leftarrow \theta$)。这种硬更新策略是 Double DQN 中常用的目标网络更新方式。

回合结束后，记录该回合的各项统计数据，包括消除行数、得分、总奖励、步数以及当前的 ε 值，并输出到控制台，以便后续分析。同时，每 SAVE_INTERVAL 回合，模型检查点将会定期保存 (torch.save)。

2.6 奖励函数

奖励函数的设计对强化学习 Agent 的行为塑造尤为重要。本研究中，奖励函数在 Tetris 类的 step 方法内部定义，对于每轮游戏结束前的操作步，奖励计算为:

$$\text{reward} = 1 + (\text{lines_cleared})^2 \times \text{self.width}$$

其中，lines_cleared 是在当前步消除的行数（0, 1, 2, 3 或 4），self.width 为棋盘宽度 (10)。对于导致游戏结束的最后一步，奖励为固定的常数：

$$\text{reward} = -10$$

奖励函数的设计蕴含着几个重要思考。首先，每存活一步（即游戏未结束）Agent 都会获得 +1 的基础生存奖励，这将会鼓励其设法延长游戏时间。其次，行消除奖励部分采用了消除行数的平方项 lines_cleared^2。非线性设计强烈激励 Agent 学习能够一次性消除多行的策略，我们可以看到，四消（Tetris）的奖励（$1 + 4^2 \times 10 = 161$）远高于消除单行（$1 + 1^2 \times 10 = 11$）。有助于引导 Agent 学习更高级的策略，如尝试搭建用于四消的结构。使用平方项并乘以棋盘宽度 self.width 进行缩放，在早期学习过程中提供更强的引导信号。最后，对导致游戏结束的动作施加一个显著的负奖励 (-10)，明确告知 Agent 应该避免这种情况。综上，该奖励函数结合了生存激励和对核心目标（消除行，尤其是多行）的强力引导，同时对最终失败进行惩罚。

2.7 实现工具

本研究的实现主要依赖 Python 3 编程语言以及一系列核心库：`PyTorch` 作为构建和训练深度神经网络（Q 网络）的基础框架，`Pygame` 用来创建和渲染俄罗斯方块的游戏环境。训练过程中的各项指标（消行数、得分、奖励、步数等）的记录通过控制台输出，训练结果可视化则由我们手工绘制完成。此外，`collections.deque` 实现了经验回放缓冲区的功能，而 `random` 模块则负责随机数的生成，如在 ε-greedy 策略中进行随机动作选择以及生成符合”7-bag” 规则的积木块序列。

3 实验与结果

本章将详细介绍评估 Agent 学习效果的实验配置，并对训练过程中收集的数据进行分析，重点关注 Agent 学习曲线与性能变化趋势。实验结果主要来源于控制台日志输出及训练过程数据的统计，并通过训练曲线图和点状图进行可视化呈现。

3.1 实验设置

Agent 的训练总共进行了 `EPOCHS = 3000` 个回合 (episodes)，每个回合从空白的棋盘状态开始，直至游戏依照规则结束为止。训练过程在 Google Colab 云计算环境中执行，并利用了 T4 GPU 进行计算加速以缩短训练时间。

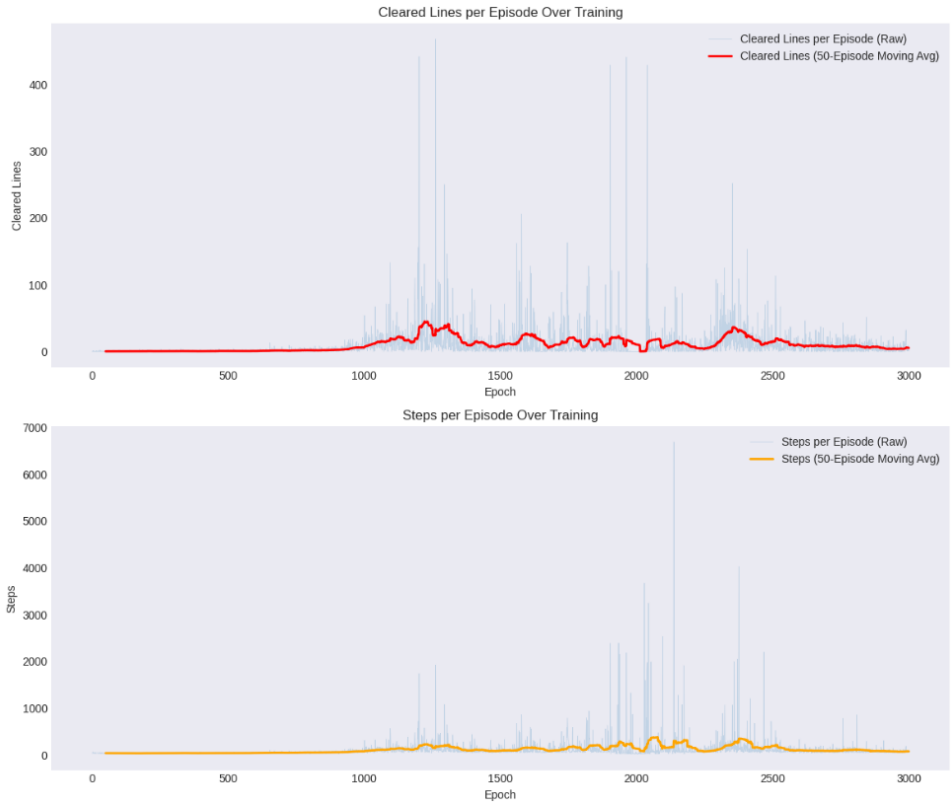
核心的强化学习算法采用了 Double DQN。Agent 的 Q 网络是一个包含三个全连接隐藏层（每层 64 个神经元，使用 ReLU 激活函数）的多层感知机 (MLP)，其输入为前述定义的 4 维状态特征向量。

实验同时设置了一系列关键的超参数：随机种子 `SEED` 固定为 `42`，以保证实验的可复现性。未来奖励的折扣因子 `GAMMA` 设置为 `0.99`，促使 Agent 关注长远目标。经验回放缓冲区中采集的批次 `BATCH_SIZE` 设置为 `512` 条，经验回放缓冲区 `replay_memory` 的最大容量为 `30,000` 条经验。Epsilon-Greedy 探索策略的参数设置为：初始探索率 `INITIAL_EPSILON` `1.0`，最终探索率 `FINAL_EPSILON` `0.001`，探索率衰减回合数 `DECAY_EPOCHS` 为 `1500`，以期平衡 Agent 探索未知（尝试新的动作）和利用已知（很具当前学到的知识选择最优动作）。优化器（Optimizer）选用 Adam，学习率（Learning rate）设定为 `1e-4`。目标 Q 网络更新频率 `update_target_rate` 设置为每 `10,000` 个总训练步数更新一次目标网络参数。

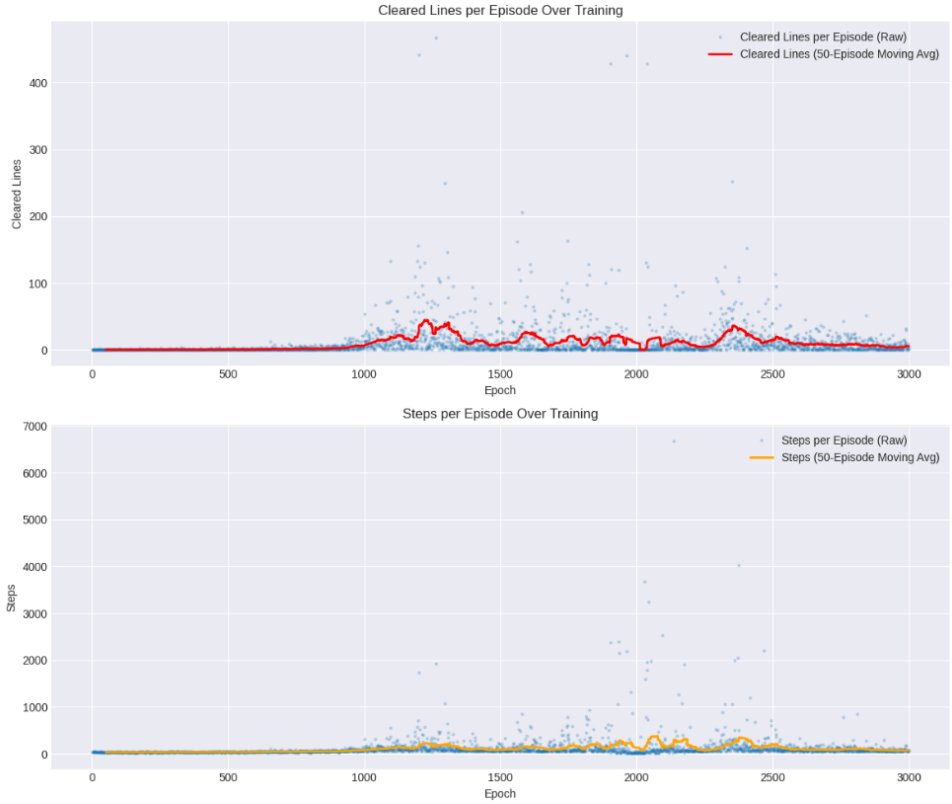
为了评估 Agent 的学习进展和最终性能，在训练过程中主要记录了以下指标：每回合消除总行数，每回合的消行数（Lines）、得分（Score）、奖励（Reward）、步数（Steps）、回合结束时的衰减率（Epsilon）。同时，为加速训练，Pygame 的图形渲染功能在训练期间被禁用 (`RENDER = False`)。

3.2 学习数据分析

对 3000 个训练回合的表现数据进行分析，能够清晰展现 Agent 的学习过程。图 3.1 综合展示了训练过程中每回合消除行数和步数的总体趋势（训练曲线图）以及单回合的表现分布（训练点状图）。以下结合控制台日志和训练曲线进行阐述。



(a) 训练曲线图



(b) 训练点状图

图 3.1　模型训练结果

3.2.1 每回合消除行数分析

每回合消除的行数是衡量俄罗斯方块 Agent 性能的核心指标。如图 3.1a 所示，经过平滑处理的训练曲线清晰地展示了消除行数的平均表现随训练回合数的提升趋势。在训练初期（约前 500 回合），Agent 的消行数较低，平均在个位数到几十行之间，这是由于 Agent 尚未学习到有效的策略，主要依靠随机探索。这一阶段在图 3.1b 的点状图中表现为密集分布在较低消除行数的区域。

随着训练的深入，尤其在 500 至 1500 回合之间，Agent 的性能取得了显著进步，平均消行数在图 3.1a 中呈现快速上升趋势。图 3.1b 也清晰地显示，在这一阶段，开始出现单回合消行数远超平均水平的“高分”离散点。训练日志记录证实了这一观察：

- 第 1095 回合：消除 133 行
- 第 1198 回合：消除 156 行
- 第 1201 回合：消除 442 行
- 第 1262 回合：达到最高纪录 468 行

这些高分回合的出现，特别是在探索率 ε 逐渐降低、Agent 更多依赖习得策略的阶段，表明 Q 网络已经开始学习到高效的消行技巧，例如搭建能够进行多行消除（尤其是四消）的棋盘结构。图 3.1b 中的高数值离散点直观地体现了 Agent 在特定回合达到的峰值性能。

在训练后期（1500 回合之后），随着 ε 降至 0.001 并保持稳定，Agent 的平均消行性能在图 3.1a 中趋于平台期或增长放缓，但仍然维持在较高的水平。尽管单回合表现（图 3.1b 中的点）存在波动，高分回合仍然偶尔出现：

- 第 1903 回合：消除 429 行
- 第 1963 回合：消除 441 行

这种波动性是强化学习训练中常见的现象，可能与经验回放采样、环境的随机性（积木块出现顺序）以及 Agent 在探索和利用之间的权衡有关。然而，能够持续达到超过 400 行的表现，充分证明了 Agent 学习到了稳健且高效的俄罗斯方块策略。

3.2.2 每回合步数分析

每回合的游戏步数与生存时间直接相关，也是衡量 Agent 游戏能力的重要指标。如图 3.1a 所示，步数训练曲线与消行曲线呈现出高度相似的总体上升趋势，两者通常正相关。训练初期，Agent 由于生存时间短，每回合步数普遍偏低（多数小于 100 步），这在图 3.1b 的低步数区域有直观体现。

随着 Agent 学习到更好的策略，避免快速游戏结束，每回合的步数显著增加。在性

能提升阶段和高分回合中，步数也随之提高，这在图 3.1b 中表现为高消行数点通常对应较高的步数：

- 第 1201 回合（442 行）：持续 1739 步
- 第 1262 回合（468 行）：持续 1919 步

然而，训练日志也揭示了在训练后期出现的一些特殊情况，即在消行数不高（甚至为 0）的回合中，步数却异常增加：

- 第 1931 回合（0 行）：持续 1066 步
- 第 1937 回合（0 行）：持续 2150 步
- 第 1979 回合（0 行）：持续 1323 步
- 第 2137 回合（29 行）：持续 6686 步（最高步数记录）

这些回合在图 3.1b 中表现为一些位于较低消行数但具有非常高步数值的离散点。它们的出现可能表明 Agent 在某些情况下学会了较为保守的策略，倾向于维持游戏状态（尽管未能大量消行）而不是冒险进行可能导致游戏结束的操作。这种保守策略虽然未能带来大量消行，却极大地延长了游戏的持续时间，从而可能通过每步的基础生存奖励 (+1) 累积总奖励。从 Agent 学习生存的角度看，这也是其策略优化的一部分，它在某些情境下优先考虑延长游戏时间而非立即寻求高回报。

3.2.3 其他训练数据分析

训练日志中记录的 Epsilon 值验证了探索策略的实施符合预期。在训练的前 1500 个回合中，Epsilon 值从 1.0 线性衰减至 0.001。这意味着 Agent 在训练初期以完全随机的方式探索各种可能的动作，而在训练后期则更多地依赖其通过训练学到的 Q 值来选择能够最大化预期回报的动作，仅保留极低的概率进行随机探索，以防止陷入局部最优。探索率的成功退火是 Agent 能够从探索阶段顺利过渡到利用阶段的关键。

同时，在训练初期，由于 Q 网络对状态-动作价值的估计非常不准确，训练损失较高。随着 Agent 与环境的交互积累经验，并利用经验回放和 Double DQN 算法进行网络参数更新，Q 网络的估计逐渐趋于准确，训练损失呈现出下降的趋势。在训练后期，损失值在较低水平上小幅波动并最终趋于稳定。损失的收敛是 Q 网络学习到价值函数、成功拟合训练数据的标志。性能指标（消行数、步数）的显著提升间接表明了学习过程的有效性，即 Q 网络成功地学习到了与游戏目标相关的状态价值。Double DQN 算法的使用，理论上可以缓解标准 DQN 中价值估计过高的问题，有助于提升训练的稳定性和最终性能。

3.2.4 整体学习动态总结

综合分析上述各项指标以及图 3.1 所示的训练曲线和点状图，可以清晰地看到 Agent 在 3000 个回合的训练中成功地学习了俄罗斯方块的玩法。每回合消除行数和存活步数这两项关键性能指标的平均曲线（图 3.1a）均显示出从初始的低水平，经过约 1000-1500 回合的学习，取得了显著的提升，最终在训练后期趋于稳定并达到了较高的水平。尤其值得注意的是，Agent 学习到了进行大量消行的策略，并多次达到了超过 400 行的高分，最高纪录为 468 行，这在图 3.1b 中表现为高离散点。同时，Agent 也学会了在某些情境下采取延长游戏时长的策略，有时即使没有大量消行也能保持较长的生存步数，如图 3.1b 中的某些高步数低消行的点所示。Epsilon 探索率的成功退火保证了 Agent 从充分探索到有效利用知识的平稳过渡。预期的损失下降趋势进一步佐证学习的有效性以及 Double DQN 可能带来的稳定性增益。诸上结果表明，基于 Double DQN 和特定特征输入的 Agent 在俄罗斯方块任务上的有效性与学习能力。

4 讨论

本章将对第三章的实验结果进行深入解读，评估训练模型方法的有效性，包括 Agent 的学习效果、状态表示和奖励函数的设计，并结合实验发现，将本研究置于更广阔的深度强化学习与游戏 AI 研究背景下，进行比较与定位，探讨本方法的优势与局限性。

4.1 实验结果分析与方法有效性评估

实验结果清晰地表明，本研究实现的基于特征输入的 Double DQN Agent 成功学习了俄罗斯方块游戏。核心性能指标，尤其是每回合消除行数和存活步数，均展现了明确的学习曲线，从最初的近乎随机表现几乎无法得分，到能够长时间进行游戏并消除大量行（参见图 3.1）。Agent 不仅稳定地提高了平均表现，还在某些回合中取得了高达 468 行的优异成绩，这有力地证明了所采用的 DRL 方法在该任务上的有效性。

从学习过程来看：Agent 在训练前 1500 回合（探索率 ε 衰减期）表现出最快速的性能提升，Agent 积极试错、积累经验、修正价值估计。随着 ε 趋于稳定，Agent 更多依赖习得策略进行利用，平均性能曲线趋缓，进入平台期。即使后期单回合表现有所波动，但持续出现的高分回合表明 Agent 并未停滞，而是在利用阶段依然能够发挥出高水平。损失函数的收敛则从优化的角度印证了 Q 网络成功地拟合了状态价值函数，为稳定决策提供了基础。

本研究的核心特点之一在于采用基于领域知识的特征工程进行状态表示，而非依赖高维原始像素输入。已有的 Tetris AI 研究广泛采用了基于特征的方法。我们从俄罗斯方块的游戏机制出发，提炼出四维特征：潜在消除行数、空洞数、棋盘崎岖度和总高度。这些特征直接关联游戏的关键目标（最大化消行、维持棋盘低位、保持平整、避免空洞），具有较高的信息密度和任务相关性。实验结果表明，Agent 仅凭这些低维特征就能够学习到复杂的游戏策略，这初步验证了在领域知识丰富、可以提取有效特征的情况下，使用低维特征输入结合 DRL 方法是可行且有效的。这种方法相比处理高维像素显著降低了对 Q 网络复杂度的要求 (使用了 MLP 而非 CNN)，简化了模型结构，并可能加速了学习过程 (3000 回合内达到较高水平)。

然而，特征工程也存在内在局限。首先，像素信息中包含的更细微的模式或局部结构信息存在丢失的可能性，这些信息对于人类玩家或基于 CNN 的 Agent 可能是有参考价值的，如描述特定积木块与棋盘现存凹槽的契合度，或者特定布局所形成的潜在陷阱等。其次，这四个特征的组合是否完全捕捉了影响最优决策的棋盘信息，依然值得商

榷，四维特征没有直接编码未来待落积木块的信息（尽管 Agent 在评估动作时会考虑当前块落下的结果），而预知下一个积木对人类玩家制定长期策略至关重要。因此，与能够自主从原始数据中学习分层特征表示的端到端方法相比，特征工程依赖于人类的先验知识，可能存在信息瓶颈，或限制了 Agent 理论上能达到的性能上限。

奖励函数的设计在本研究中发挥了核心的策略引导作用。每一步的 +1 生存奖励设定，有效激励 Agent 学习如何避免游戏提前结束，将长期生存作为基础目标。而对消除行数使用平方项并乘以棋盘宽度进行放大的奖励机制，巧妙地鼓励 Agent 追求高难度、高回报的多行消除，尤其是四消。实验中出现的大量消行记录直接反映了这一非线性奖励设计在塑造 Agent 追求高难度、高回报行为方面的有效性。游戏结束时设定的显著负奖励则清晰地界定了 Agent 需要规避的最终失败状态。这种结合了生存、高效消行和避免失败的奖励结构，成功地引导 Agent 在不同目标之间找到了一种有效的平衡策略。

算法层面，本研究采用的 Double DQN[5] 对标准 DQN[4] 固有的 Q 值过高估计问题进行了修正。通过解耦动作选择和价值评估，Double DQN 能够提供更准确的价值估计，理论上能带来更稳定、收敛性更好的学习过程。本实验中观察到的平稳学习曲线，相较于文献中描述的标准 DQN 可能出现的性能崩溃现象[4]，在一定程度上得益于 Double DQN 机制的应用。

4.2 基于实验结果的方法比较与讨论

将本研究成果置于更广阔的 AI 研究视野下，结合实验结果，可以进行如下比较和讨论：

与基于特征的早期方法相比，Tetris AI 的早期研究大量依赖手工设计的启发式评估函数，并通过策略迭代或进化算法（如 Genetic Algorithms, CMA-ES ）来优化这些函数的权重。本研究同样采用了基于特征的方法，但将其与深度强化学习框架结合，利用神经网络的强大拟合能力自主学习特征组合的权重，而非直接优化预定义的线性函数，与早期方法相比，DRL 能够从经验中自主学习更复杂的策略表示。

与标准 DQN[4] 及其变体（Dueling, PER, Rainbow）[6,8-9] 比较，本研究采用 Double DQN[5]，是对标准 DQN 的改进，旨在减少价值过估，提升稳定性[5-6]。Rainbow Agent[6] 则进一步融合了 Double DQN、优先经验回放 (PER)、决斗网络 (Dueling Networks)、多步学习、分布式 RL 和噪声网络等多种先进技术。由于俄罗斯方块的奖励结构具有稀疏性和延迟性，PER[9] 可通过优先学习那些带来较大 TD 误差的经验来提高学习效率；多步学习可以使用 n-step 回报将奖励信号更快地反向传播，有助于解决奖励延迟问题；决斗网络[8] 有助于更准确评估状态价值。因此，虽然本研究证明了基础特征和 DDQN 的

有效性，集成 Rainbow 的其他关键组件[6] 极有可能进一步提升 Agent 的学习效率和最终性能上限。

进化策略[9-10]属于黑盒优化，不依赖梯度，易于并行化。本研究的 DRL 方法依赖梯度下降和价值函数学习。ES 也可以用来优化 Q 网络的参数或启发式函数的权重。虽然 ES 在某些问题上表现良好且对奖励函数形态不敏感，但通常认为其在样本效率上低于基于梯度的 DRL 方法，这可能意味着需要更多的环境交互才能达到相似的性能水平。本研究的 Agent 在 3000 回合内达到了较高的性能，显示出一定的样本效率。

启发式搜索结合评估函数，显式地探索未来步骤以做出决策。其优点在于决策过程相对可解释，性能严重依赖评估函数的质量，并且搜索深度和宽度会影响计算成本和决策质量。本研究 DRL Agent 学习隐式策略（体现在 Q 值中），决策速度通常更快。未来可以将学习到的 Q 网络作为启发式函数用于搜索，探索是否能进一步提升决策质量。

近期研究探索使用大语言型模型（如 Transformer 等）通过离线（offline）强化学习或上下文（in-context）强化学习等，处理复杂游戏（如宝可梦对战），展现了强化学习处理复杂信息和高级推理的能力。例如，来自佐治亚理工学院的华人学者构建了“宝可梦智能体”[2]，在天梯对战中与人类玩家一较高下，对战专业玩家胜率达到了 56%，该工作利用了上下文强化学习、知识增强生成（Knowledge-Augmented Generation）和一致性动作生成（Consistent Action Generation）等策略，展现了 LLM 在处理文本化的复杂游戏状态、利用外部知识以及进行高级推理决策方面的潜力。而来自德克萨斯大学奥斯汀分校的学者们则利用离线人类数据和合成自对弈数据上进行训练[3]，最终的模型 (SynRL-V2) 在与真实人类玩家的匿名在线对战中，能够在官方 PS 天梯上达到活跃玩家前 10% 的水平，该模型性能优于上述“宝可梦智能体”，并且在早期世代的长回合对战中能与强大的启发式搜索引擎 (Foul Play) 抗衡甚至超越。然而，本研究与这些基于 LLM 的 Agent 方法范式存在显著差异。我们的方法立足于在明确规则和可提取有效特征的结构化环境中，通过价值函数学习实现高效控制。而基于 LLM 的方法则倾向于利用模型强大的语言理解和生成能力，处理更复杂的、文本化的游戏信息，并在某些需要推理和策略交互的游戏中展现优势。尽管任务类型和方法基础不同，但这些研究都共同推动了 AI 在复杂游戏决策领域的发展。或可以探索将 Transformer 等先进模型架构应用于处理 Tetris 的序列状态或行动轨迹，以及借鉴 POKÉLLMON[2] 中利用外部知识的思想，为 Tetris Agent 设计提供新的视角。

5 未来工作

本研究虽已证实了基于特征输入的 Double DQN 方法在俄罗斯方块任务上的有效性，但 AI 的探索永无止境，仍有诸多方向值得进一步深入研究与改进，以期突破性能上限、提升方法普适性并深化对智能学习机制的理解。

在算法增强与比较方面，首先应该进行更为精细的消融研究，严格控制其他变量，对比标准 DQN[4] 与 Double DQN[5] 在本任务中的性能差异与学习稳定性，量化 Double DQN 的实际贡献。其次，可以考虑融合先进技术，系统地将 Rainbow[6] 中的其他有效组件逐步集成到当前 Agent 中，并进行消融研究。特别关注优先经验回放 (PER)[7] 与多步学习，鉴于俄罗斯方块中奖励的稀疏性和延迟性，可能对提高样本效率和加速学习有显著帮助。

关于状态与动作表示，可以探索更丰富的特征集，例如加入对未来待落积木块信息的编码、更精细地描述井 (well) 的分布和深度、或采用更复杂的棋盘平整度度量等，以期捕捉更多状态信息，提升性能上限。此外，也可以研究更细粒度的动作空间，例如允许 Agent 控制中间的移动和旋转步骤，但这会显著增加决策的复杂度，需要更高级的 RL 算法来处理，但可能赋予 Agent 更灵活的控制能力，使其策略更接近人类玩家的操作。

在训练与优化层面，将训练时长大幅延长，例如至数万或数十万级别，以观察 Agent 性能是否能进一步提升并达到更高的收敛水平。同时，应采用系统性的超参数优化方法 (如网格搜索、随机搜索或贝叶斯优化) 来寻找更优的参数组合，涵盖学习率、网络结构、折扣因子等。奖励函数的设计也是一个值得探索的方面。设计并实验不同形式的奖励函数，例如，引入对空洞、高列或其他不良棋盘结构的惩罚项，或采用更精细的奖励塑形方法，以更平滑地引导 Agent 学习期望的行为，并研究不同奖励设计对最终策略风格的影响。

最后，对实验结果还可以采用进一步分析，或采用定性研究的方法，通过可视化其游戏过程来观察典型的策略模式、优势和潜在缺陷 (例如，是否擅长搭建用于消除 Tetris 的结构，是否容易在特定情况下产生空洞)。此外，还可以进行特征重要性分析，以量化评估本研究使用的四个状态特征各自对于最终决策的贡献度。

通过在上述方向上的持续深入研究，不仅能够持续提升俄罗斯方块 AI Agent 的水平，挑战更高难度的人类玩家表现，更重要的是，这些探索将为理解和构建能够处理复杂、随机和长期依赖性问题的智能体提供宝贵的理论见解和实践经验。

参考文献

[1] Da Col, G., & Teppan, E. C. (2019). Heuristic search for Tetris: A case study. In K. Arai, R. Bhatia, & S. Kapoor (Eds.), Intelligent Computing. CompCom 2019. Advances in Intelligent Systems and Computing (Vol. 997). (pp. 410–423).Springer, Cham. https://doi.org/10.1007/978-3-030-22871-2_28

[2] Hu, S., Huang, T., & Liu, L. (2024). PokéLLMon: A Human-Parity Agent for Pokémon Battles with Large Language Models. arXiv. https://doi.org/10.48550/arXiv.2402.01118

[3] Grigsby, J., Xie, Y., Sasek, J., Zheng, S., & Zhu, Y. (2025). Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers. arXiv. https://doi.org/10.48550/arXiv.2504.04395

[4] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. https://doi.org/10.1038/nature14236

[5] Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. In Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI’16) (pp. 2094–2100). AAAI Press. arXiv. http://doi.org/10.48550/arXiv.1509.06461

[6] Hessel, M., Modayil, J., Van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., & Silver, D. (2017). Rainbow: Combining improvements in deep reinforcement learning. arXiv. https://arxiv.org/abs/1710.02298

[7] Horgan, D., Quan, J., Budden, D., Barth-Maron, G., Hessel, M., Van Hasselt, H., & Silver, D. (2018). Distributed prioritized experience replay. arXiv. https://arxiv.org/abs/1803.00933

[8] Wang, Z., Schaul, T., Hessel, M., Van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. arXiv. https://doi.org/10.48550/arXiv.1511.06581

[9] Salimans, T., Ho, J., Chen, X., Sidor, S., & Sutskever, I. (2017). Evolution strategies as a scalable alternative to reinforcement learning. arXiv. https://doi.org/10.48550/arXiv.1703.03864

[10] Boumaza, A. (2011). Designing artificial Tetris players with evolution strategies and racing. In Proceedings of the 13th Annual Conference Companion on Genetic and Evolutionary Computation (GECCO’11) (pp. 117–118). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2001858.2001925

致谢

我依然清晰记得三月三十号，那个充满期待的夜晚，我单击“runcell”，注视着控制台一行一行输出的训练日志。然而，训练结果远未如预期，人工智能的表现也并非我想象中那般“聪明”。之后继续迭代了六版程序，它们几乎都是徒劳的尝试——每次一千局、两千局的模拟运行，最终屏幕上呈现的，始终是成群结队的“0 行消除”和触目惊心的负数得分。面对这些冰冷的数字，沉重的叹息与尴尬的苦笑是我最直接的反应。正是在“暂停”与“重启”的循环里，我开始慢慢学会消化无力感，逐渐成长为一名情绪稳定的大人。

正当我感觉研究陷入僵局、几乎要钻进死胡同时，转机出现。来自计算机专业朋友们给出了及时的想法贡献与技术指导，同时我放弃“闭门造车”，转向 GitHub 上的前辈们学习宝贵经验。正是他们的启发与帮助，我才鼓起勇气推翻之前的方案，并在第七版代码中，见证了百行方块被稳定消除的惊喜。

同时，我也清醒地认识到，囿于现实条件（算力有限、调参困难）和自身的技术瓶颈，我未能如设想中那样，对代码进行更进一步地优化。最终的成果或许并不完美，但已是我当下能力所及的最好呈现。此外，正因为将绝大部分精力几乎“孤注一掷”地投入到了代码的实现与调试，我必须坦诚，论文文本的打磨确实显得有些仓促与不足，恳请审阅的老师们多加包涵。

这份论文的产出，离不开许多人的支持与帮助。在此，我特别感谢他们：

首先，感谢我的导师鲜于波老师。他不仅在学术上给予了我细致入微的指导，更为我的职业发展提供了宝贵的建议。当我无数次深陷 bug 泥潭、逻辑混乱，带着问题“找上门来”的时候，导师总能以他的智慧和包容，为我疏导迷津，告诉我“慢慢来”。

感谢最初那个小小的火焰。那是去年十月份，我使用 AI 工具独立完成了一套个性化的俄罗斯方块游戏项目。正是这次尝试，使我发现：一个并不擅长编程的人，在这个时代，也能借助大语言模型的力量，实现自己的程序设计。这也是这篇论文的最初起点，也感谢那个勇敢尝试的自己。

感谢我的室友们，在那段“兵荒马乱”的日子里，她们的存在本身就是一种治愈。在我为雅思、申请、考试、论文焦头烂额的时候，她们总会以玩笑的方式，巧妙地帮助我放松紧绷的神经。

感谢我的同事们在我兼顾学业与工作期间给予的关心与鼓励，以及我的 mentor 和 leader 在时间安排上的理解和支持，让我在双重压力下依然保持热情与活力。

感谢给予我片刻喘息与心灵慰藉的美好存在。感谢熊德龙 105 和智慧琴房的钢琴，

它们承载了我所有的情绪表达。感谢我的狗狗雪饼，它用最简单纯粹的方式教会我责任与承担，让我相信世界上存在着无条件的爱。不在家的日子里，感谢学校里那些自由自在的小动物们，特别是学校北门那两只被我唤作“金角银角”的猫咪，感谢它们接受我的投喂，并对我表示友好：贴贴蹭蹭、打滚儿伸懒腰。观察它们安静吃饭的样子，便能感受到片刻的宁静与柔软。

感谢那些在我独处的时刻，给予我精神滋养和呼吸空间的文艺作品。英剧《布莱克书店》、《IT 狂人》、《伦敦生活》的黑色幽默，美剧《老友记》的温暖陪伴，以及无数让人动容的电影、话剧、音乐剧，还有萨冈的文字与 Olivia Rodrigo、OneRepublic、Coldplay 的音乐等，它们是我的精神食粮，也提醒着我生活的意义所在。

最后，感谢在我在焦虑、内耗、自我怀疑最严重的时期，收到了香港中文大学（深圳校区）的录取通知。收到了来自梦校的认可，这份突如其来的巨大的肯定与喜悦，令我兴奋不已，也为我带来了莫大的慰藉和继续前行的动力。

Tomorrow is another day.